Matrix Factorization and Genre-Based Recommender Systems on MovieLens-32M

Joel Gedeon Asa Mapa [1]

Abstract

Building a recommender system for tens of millions of ratings requires both a solid statistical model and fast data structures. In this paper, we build and study matrix factorization models trained with Alternating Least Squares (ALS) on the full MovieLens-32M dataset (32,000,204 ratings from 200,948 users and 84,432 movies), as well as a smaller 5-million-rating subset. To store the ratings efficiently in memory, we use two Compressed Sparse Row (CSR) matrices: one indexed by user and one indexed by movie. This setup uses about 513 MB of RAM and allows us to look up all ratings for any user or movie in constant time. By parallelizing the ALS updates across CPU threads and solving the small linear systems in-place with Cholesky decomposition, our training loop runs nearly 70 times faster than batched array solvers. We begin by exploring the rating distribution and power-law behavior of users and movies. Next, we compare a simple bias-only model against full matrix factorization with user and movie trait vectors, showing how regularization and the number of latent dimensions K affect test error and overfitting. We also visualize 2D movie embeddings, generate personalized recommendations for a new user who likes *The Lord of the Rings*, and identify the most and least polarizing movies using vector lengths. Finally, we extend the model by adding movie genre information into the prior distribution of movie vectors, which noticeably improves predictions for new and rarely rated movies. Our source code is publicly available at https://github.com/joelinator/movielens-32m-recommender.

[1]African Institute for Mathematical Sciences (AIMS) South Africa, 6 Melrose Road, Muizenberg 7975, Cape Town, South Africa. Correspondence to: Joel Gedeon Asa Mapa <joel@aims.ac.za>.



1. Introduction

Online platforms such as streaming services and online stores rely on recommender systems to help users find items they might enjoy among tens of thousands of choices (Koren et al., 2009; Harper & Konstan, 2015). One of the most effective approaches to this problem is collaborative filtering, which predicts what a user will like based on past ratings from many users, without needing detailed descriptions of the items themselves.

When working with large datasets like MovieLens-32M, which has over 32 million ratings across 200,948 users and 84,432 movies, storing the ratings in a standard two-dimensional table is not practical. A full table of size $200{,}948 \times 84{,}432$ using 32-bit floats would take 67.8 GB of memory, even though 99.81% of the entries are empty because most users only rate a small fraction of the catalog.

Matrix factorization handles this sparsity by representing each user m and each movie n with a short vector of K numbers, $u_m \in \mathbb{R}^K$ and $v_n \in \mathbb{R}^K$, called latent trait vectors (Mnih & Salakhutdinov, 2007; Koren et al., 2009). A rating is then predicted as the dot product $u_m^\top v_n$ plus baseline user and movie biases. To learn these parameters from observed ratings, two main algorithms are commonly used: Stochastic Gradient Descent (SGD) and Alternating Least Squares (ALS) (Zhou et al., 2008). ALS is especially attractive for large datasets for three reasons:

1. It does not require tuning a learning rate, because each update step has an exact closed-form formula.

2. The regularized loss function is guaranteed to decrease (or stay the same) at every single step.

3. When all movie vectors are held fixed, each user vector can be updated independently of all other users. Likewise, when user vectors are fixed, all movie vectors can be updated in parallel across CPU cores.

1.1. Problem Statement and Structure of the Paper

Our goal is to predict unseen movie ratings accurately at scale while also understanding what the model learns and where it struggles. Following a clear separation between data exploration, modeling, and experimental evaluation, the rest of this paper is organized as follows:

1. **Data Representation and Exploration (Section 2):** We show how to store 32 million ratings compactly in two CSR matrices ($\mathcal{S}_{\text{user}}$ and $\mathcal{S}_{\text{movie}}$) so that both user updates and movie updates run in constant lookup time, and we analyze the rating histogram and user/movie power-law distributions.

2. **Models and ALS Optimization (Section 3):** We formulate all three of our models and derive their Alternating Least Squares (ALS) update steps in one place: (i) the baseline bias-only model (Section 3.1), (ii) the matrix factorization model with user and movie trait vectors U and V (Section 3.2), and (iii) the genre-regularized matrix factorization model with altered loss function and three-step ALS updates (Section 3.3), together with our parallel in-place Cholesky solver (Section 3.4).

3. **Experimental Results (Section 4):** We present a comprehensive evaluation across both MovieLens-5M and the full MovieLens-32M dataset, including our train/test split, bias-only and matrix factorization RMSE and monotonic loss curves, choice of training iterations, hyperparameter tuning (λ, γ, τ), choice of best latent dimension K and diminishing returns, runtime speedups on all 32 million ratings, 2D movie and genre embeddings, *Lord of the Rings* dummy-user recommendations, and most/least polarizing movies.

2. Data Representation and Exploratory Analysis

2.1. Dual Sparse Matrix Structure

Let M be the number of users and N be the number of movies. The dataset consists of observed ratings $r_{m,n} \in \{0.5, 1.0, \ldots, 5.0\}$ for user–movie pairs $(m, n) \in \Omega$, where Ω is the set of all observed interactions. For each user m, we write $\Omega_m = \{n : (m, n) \in \Omega\}$ for the set of movies rated by user m. Similarly, for each movie n, we write $\Omega^n = \{m : (m, n) \in \Omega\}$ for the set of users who rated movie n. The total number of ratings is $|\Omega| = \sum_{m=1}^{M} |\Omega_m| = \sum_{n=1}^{N} |\Omega^n|$.

During ALS training, updating user parameters requires looping over all movies rated by each user (Ω_m), while updating movie parameters requires looping over all users who rated each movie (Ω^n). If we only stored the data grouped by user, updating the movies would require either scanning the entire dataset repeatedly or jumping randomly in memory. Using Python lists of lists or dictionaries also uses too much memory and slows down CPU loops.

Table 1. Summary statistics of the MovieLens-5M subset and the full MovieLens-32M dataset.

Statistic	ML-5M	ML-32M
Number of Users (M)	31,205	200,948
Number of Movies (N)	48,708	84,432
Total Ratings ($\lvert\Omega\rvert$)	5,000,000	32,000,204
Train Ratings ($\lvert\Omega^{\text{TRAIN}}\rvert$)	4,000,446	25,604,307
Test Ratings ($\lvert\Omega^{\text{TEST}}\rvert$)	999,554	6,395,897
Matrix Density	0.3290%	0.1886%
Mean Rating μ / Std. σ	3.540 / 1.060	3.540 / 1.059
User Ratings Min / Median	16 / 73.0	20 / 73.0
User Ratings Mean / Max	160.2 / 9,577	159.2 / 33,332
Movie Ratings Min / Median	1 / 3.0	1 / 5.0
Movie Ratings Mean / Max	102.7 / 16,041	379.0 / 102,929
User Power-Law Slope $\hat{\alpha}_{\mathcal{U}}$	1.90	1.90
Movie Power-Law Slope $\hat{\alpha}_{\mathcal{I}}$	1.51	1.48

To make both half-steps fast, we store the ratings in two Compressed Sparse Row (CSR) structures:

- **User-Indexed CSR Matrix ($\mathcal{S}_{\text{user}}$):** Uses three flat arrays: an offset array $p^{(u)}$ of length $M + 1$, a movie-index array of length $|\Omega|$, and a rating array of length $|\Omega|$. For any user m, their rated movie IDs and ratings sit side-by-side in memory between indices $p_m^{(u)}$ and $p_{m+1}^{(u)}$.
- **Movie-Indexed CSR Matrix ($\mathcal{S}_{\text{movie}}$):** Uses the same layout transposed by movie: an offset array $p^{(v)}$ of length $N + 1$, a user-index array of length $|\Omega|$, and a rating array of length $|\Omega|$. For any movie n, all users who rated it sit contiguously between indices $p_n^{(v)}$ and $p_{n+1}^{(v)}$.

Using 32-bit integers for indices and offsets and 32-bit floats for ratings, storing both CSR views takes $16|\Omega| + 4(M + N + 2)$ bytes. As shown in Table 1, for all 32,000,204 ratings in MovieLens-32M, both structures together take only 513.1 MB of RAM (over 130 times smaller than a dense matrix) and load from disk in 4.23 seconds. Because the ratings for any user or movie sit next to each other in memory, the CPU can read them sequentially without pointer-chasing, which makes our training loops much faster.

2.2. Rating Distribution and Power-Law Behavior

Figure 1(a) shows the distribution of star ratings from 0.5 to 5.0 on both MovieLens-32M and the 5M subset. The two datasets have almost identical rating proportions (within 0.02% on every bar). Two simple patterns stand out:

1. **Users Tend to Give High Ratings:** The average rating

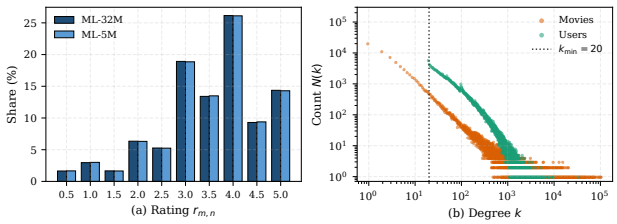


Figure 1. Exploratory plots of the MovieLens-32M and 5M datasets. **(a)** Percentage of ratings at each star value from 0.5 to 5.0, showing a peak at 4.0 stars and lower counts on half-star values. **(b)** Log-log plot of the number of movies and users with k ratings on MovieLens-32M, showing clear power-law tails and the minimum cutoff of $k_{\min} = 20$ ratings per user.

is $\mu = 3.540$ (with standard deviation $\sigma = 1.059$), which is well above the middle of the scale (2.75). The most common rating is 4.0 stars (26.15% of all ratings, or 8,367,654 ratings), followed by 3.0 stars (18.92%) and 5.0 stars (14.36%). Very low ratings (0.5, 1.0, and 1.5 stars combined) make up only 6.26% of the data. This happens because people mostly choose to watch movies they expect to like (Koren et al., 2009).

2. **Whole Stars Are Used More Than Half Stars:** Every half-star rating $(0.5, 1.5, 2.5, 3.5, 4.5)$ appears much less often than the whole-star ratings next to it. For instance, 3.5 stars is 13.41% of the data, whereas 3.0 is 18.92% and 4.0 is 26.15%. There are two reasons for this: MovieLens only added half-star ratings in 2003 (older ratings were whole numbers only), and many users naturally prefer round numbers when rating quickly.

Figure 1(b) plots the degree distribution on a log-log scale: the horizontal axis is the number of ratings k, and the vertical axis is how many movies or users have exactly k ratings. Both curves follow a straight line over several orders of magnitude on the log-log plot, which is the signature of a power law $P(k) \propto k^{-\alpha}$ (Clauset et al., 2009). However, movies and users behave very differently on the left side of the plot:

- **Long Tail of Unpopular Movies:** Movie rating counts range from $k = 1$ all the way to $k_{\max} = 102{,}929$ (*The Shawshank Redemption (1994)*). Out of 84,432 movies in MovieLens-32M, 16,895 movies have only 1 rating, and half of all movies have 5 ratings or fewer (median = 5.0), even though the average movie has 379.0 ratings. Using the standard maximum-likelihood formula for the power-law exponent (Clauset et al., 2009) on the tail ($k_{\text{tail}} = 50$),

$$\hat{\alpha} = 1 + |\mathcal{T}| \left(\sum_{i \in \mathcal{T}} \ln \frac{k_i}{k_{\text{tail}} - 0.5} \right)^{-1}, \qquad (1)$$

gives a slope of $\hat{\alpha}_{\mathcal{I}} = 1.48$ for movies. A small group of blockbuster movies gets most of the ratings, while most of the catalog sits in the sparse long tail.

- **Sharp Cutoff at $k_{\min} = 20$ for Users:** Unlike movies, the user curve in Figure 1(b) stops suddenly at a vertical line at $k_{\min} = 20$ on MovieLens-32M (minimum 20, median 73.0, mean 159.2, maximum 33,332). This happens because the creators of the MovieLens dataset removed all users with fewer than 20 ratings before releasing the data (Harper & Konstan, 2015). Above $k = 20$, user activity follows a power law with slope $\hat{\alpha}_{\mathcal{U}} = 1.90$. Because of this rule, every user in the dataset has at least 20 ratings (around 16 in the training set), whereas thousands of movies have only 1 or 2 ratings. This makes cold-start much harder on the movie side than on the user side, which motivates our genre model in Section 3.3.

3. Models and Optimization

In this section, we present all three models developed in this paper and derive their Alternating Least Squares (ALS) optimization steps: (1) the baseline bias-only model, (2) the matrix factorization model with user and movie trait vectors U and V, and (3) the genre-regularized matrix factorization model that incorporates movie genre metadata into the item prior, followed by our parallel in-place Cholesky implementation.

3.1. Bias-Only Model

Before learning latent vectors, it is helpful to build a simple baseline model that captures general rating habits. Some users are generous and give high scores to almost everything ($a_m > 0$), while others are strict critics ($a_m < 0$). Likewise, popular classics get high scores from most viewers ($b_n > 0$), while weak movies get low scores ($b_n < 0$). The bias-only model predicts:

$$\hat{r}_{m,n} = \mu + a_m + b_n, \qquad (2)$$

where μ is the average training rating ($\mu = 3.5404$ on MovieLens-32M), $a_m \in \mathbb{R}$ is the bias of user m, and $b_n \in \mathbb{R}$ is the bias of movie n.

From a Bayesian viewpoint, we assume Gaussian rating noise $r_{m,n} \sim \mathcal{N}(\mu + a_m + b_n, \lambda^{-1})$ with precision $\lambda > 0$, and zero-mean Gaussian priors $a_m \sim \mathcal{N}(0, \gamma^{-1})$ and $b_n \sim \mathcal{N}(0, \gamma^{-1})$ with regularization strength $\gamma > 0$. Taking

the negative log-posterior (dropping constants) gives the regularized squared-error loss function:

$$\mathcal{L}(a,b) = \frac{\lambda}{2} \sum_{(m,n)\in\Omega^{\text{train}}} (r_{m,n} - \mu - a_m - b_n)^2 + \frac{\gamma}{2}\sum_{m=1}^{M} a_m^2 + \frac{\gamma}{2}\sum_{n=1}^{N} b_n^2. \tag{3}$$

To minimize $\mathcal{L}(a,b)$ with Alternating Least Squares, we alternate between fixing b to solve for a, and fixing a to solve for b. Taking the derivative of $\mathcal{L}(a,b)$ with respect to a_m and setting it to zero gives the exact update for user m:

$$a_m \leftarrow \frac{\lambda \sum_{n\in\Omega^{\text{train}}_m} (r_{m,n} - \mu - b_n)}{\lambda|\Omega^{\text{train}}_m| + \gamma}. \tag{4}$$

Similarly, holding a fixed and setting $\partial\mathcal{L}/\partial b_n = 0$ gives the exact update for movie n:

$$b_n \leftarrow \frac{\lambda \sum_{m\in\Omega^{n}_{\text{train}}} (r_{m,n} - \mu - a_m)}{\lambda|\Omega^{n}_{\text{train}}| + \gamma}. \tag{5}$$

Notice how the regularization parameter γ works in Equation (5): if we write $\bar{e}_n = \frac{1}{|\Omega^n_{\text{train}}|}\sum_{m\in\Omega^n_{\text{train}}}(r_{m,n} - \mu - a_m)$ for the average residual on movie n, then

$$b_n = \left(\frac{|\Omega^n_{\text{train}}|}{|\Omega^n_{\text{train}}| + \gamma/\lambda}\right)\bar{e}_n. \tag{6}$$

When a movie has thousands of ratings ($|\Omega^n_{\text{train}}| \gg \gamma/\lambda$), the fraction is close to 1 and $b_n \approx \bar{e}_n$. When a movie has only 1 or 2 ratings, the denominator pulls b_n closer to 0 so the model does not trust a single noisy rating too much.

We also note a useful sanity check for our dual CSR data structures: computing the sum of squared errors by looping over users ($\mathcal{S}_{\text{user}}$) must give the exact same number as looping over movies ($\mathcal{S}_{\text{movie}}$), because both sums add up the exact same set of training ratings Ω^{train} in a different order:

$$\begin{aligned}\mathcal{L}_{\mathcal{U}}(a,b) &= \frac{\lambda}{2}\sum_{m=1}^{M}\sum_{n\in\Omega^{\text{train}}_m}(r_{m,n} - \hat{r}_{m,n})^2 + \mathcal{R}(a,b)\\ &= \frac{\lambda}{2}\sum_{n=1}^{N}\sum_{m\in\Omega^{n}_{\text{train}}}(r_{m,n} - \hat{r}_{m,n})^2 + \mathcal{R}(a,b)\\ &= \mathcal{L}_{\mathcal{I}}(a,b),\end{aligned} \tag{7}$$

where $\hat{r}_{m,n} = \mu + a_m + b_n$ and $\mathcal{R}(a,b) = \frac{\gamma}{2}(\|a\|_2^2 + \|b\|_2^2)$.

3.2. Matrix Factorization with User and Movie Trait Vectors

The bias-only model treats all users as if they agree on which movies are good or bad, differing only by a constant offset. To model personal taste (for example, liking sci-fi vs. romance), we add a K-dimensional latent vector $u_m \in \mathbb{R}^K$ for each user and $v_n \in \mathbb{R}^K$ for each movie (Mnih & Salakhutdinov, 2007; Koren et al., 2009):

$$\hat{r}_{m,n} = \mu + a_m + b_n + u_m^\top v_n, \quad u_m, v_n \in \mathbb{R}^K. \tag{8}$$

We place independent zero-mean Gaussian priors $u_m \sim \mathcal{N}(\mathbf{0}, \tau^{-1}I_K)$ and $v_n \sim \mathcal{N}(\mathbf{0}, \tau^{-1}I_K)$ on the trait vectors with regularization parameter $\tau > 0$. The full regularized loss function over $\Theta = \{a, b, U, V\}$ is:

$$\begin{aligned}\mathcal{L}(\Theta) = \frac{\lambda}{2}\sum_{(m,n)\in\Omega^{\text{train}}}\left(r_{m,n} - \mu - a_m - b_n - u_m^\top v_n\right)^2 \\ + \frac{\gamma}{2}\left(\sum_{m=1}^{M} a_m^2 + \sum_{n=1}^{N} b_n^2\right) \\ + \frac{\tau}{2}\left(\sum_{m=1}^{M}\|u_m\|_2^2 + \sum_{n=1}^{N}\|v_n\|_2^2\right).\end{aligned} \tag{9}$$

Because $u_m^\top v_n$ multiplies two unknowns together, $\mathcal{L}(\Theta)$ is not convex when U and V vary at the same time. However, if we hold the movie parameters (b, V) fixed, $\mathcal{L}(\Theta)$ becomes a simple quadratic function of (a_m, u_m) for each user m separately. Setting the derivatives with respect to a_m and u_m to zero (see Section A for the step-by-step derivation) gives the user updates:

$$a_m \leftarrow \frac{\lambda \sum_{n\in\Omega^{\text{train}}_m}\left(r_{m,n} - \mu - b_n - u_m^\top v_n\right)}{\lambda|\Omega^{\text{train}}_m| + \gamma}, \tag{10}$$

$$u_m \leftarrow \left(\lambda\sum_{n\in\Omega^{\text{train}}_m} v_n v_n^\top + \tau I_K\right)^{-1}\left(\lambda\sum_{n\in\Omega^{\text{train}}_m}\tilde{r}^{(u)}_{m,n} v_n\right), \tag{11}$$

where $\tilde{r}^{(u)}_{m,n} = r_{m,n} - \mu - a_m - b_n$ is the rating residual after subtracting the biases.

In the second half of each ALS iteration, we hold (a, U) fixed and update each movie's bias b_n and trait vector v_n using the movie-indexed CSR matrix $\mathcal{S}_{\text{movie}}$:

$$b_n \leftarrow \frac{\lambda \sum_{m\in\Omega^{n}_{\text{train}}}\left(r_{m,n} - \mu - a_m - u_m^\top v_n\right)}{\lambda|\Omega^{n}_{\text{train}}| + \gamma}, \tag{12}$$

$$v_n \leftarrow \left(\lambda\sum_{m\in\Omega^{n}_{\text{train}}} u_m u_m^\top + \tau I_K\right)^{-1}\left(\lambda\sum_{m\in\Omega^{n}_{\text{train}}}\tilde{r}^{(v)}_{m,n} u_m\right), \tag{13}$$

where $\tilde{r}^{(v)}_{m,n} = r_{m,n} - \mu - a_m - b_n$. Because every coordinate step finds the exact minimum for that block of variables, the total loss $\mathcal{L}(\Theta)$ is guaranteed to go down at every iteration.

3.3. Adding Movie Genres: Altered Loss Function and ALS Steps

A clear weakness of standard matrix factorization appears when a movie is brand new ($|\Omega^n_{\text{train}}| = 0$) or has only a few ratings: because the prior on v_n is centered at zero, Equation (13) sets $v_n = \mathbf{0}$ for any unrated movie. As a result, the dot product $u_m^\top v_n$ becomes zero, and the model cannot personalize predictions for new movies at all. To fix this, we incorporate the movie genre metadata directly into our model (Agarwal & Chen, 2009).

In the MovieLens catalog, each movie n is tagged with a set of genres $\mathcal{G}_n \subseteq \{1, \dots, G\}$ (for example, Adventure, Children, and Fantasy for *Jumanji (1995)*; there are $G = 20$ genres in total, including an "unlisted" category). We assign each genre $g \in \{1, \dots, G\}$ its own latent trait vector $f_g \in \mathbb{R}^K$. Instead of centering the Gaussian prior of v_n at zero, we center it at the normalized sum of the movie's genre vectors:

$$\bar{f}_n = \frac{1}{\sqrt{|\mathcal{G}_n|}} \sum_{g \in \mathcal{G}_n} f_g, \qquad v_n \,|\, F \sim \mathcal{N}\big(\bar{f}_n, \tau^{-1} I_K\big). \tag{14}$$

Dividing by $\sqrt{|\mathcal{G}_n|}$ keeps the variance of $\bar{f}_n$ constant whether a movie has 1 genre tag or 5 genre tags. Placing a Gaussian prior $f_g \sim \mathcal{N}(\mathbf{0}, \tau_f^{-1} I_K)$ on the genre vectors gives our altered loss function:

$$\begin{aligned}\mathcal{L}_{\text{genre}} = \frac{\lambda}{2} \sum_{(m,n)\in\Omega} \big(r_{mn} - \mu - a_m - b_n - u_m^\top v_n\big)^2 \\ + \frac{\gamma}{2}\left(\sum_{m=1}^{M} a_m^2 + \sum_{n=1}^{N} b_n^2\right) + \frac{\tau}{2}\sum_{m=1}^{M} \|u_m\|_2^2 \\ + \frac{\tau}{2}\sum_{n=1}^{N} \big\|v_n - \bar{f}_n\big\|_2^2 + \frac{\tau_f}{2}\sum_{g=1}^{G} \|f_g\|_2^2. \end{aligned} \tag{15}$$

Optimization Steps. Because the new penalty $\frac{\tau}{2}\sum_{n=1}^{N} \|v_n - \bar{f}_n\|_2^2$ is quadratic in both v_n and f_g, each ALS epoch cleanly splits into three alternating update steps:

1. **Step 1: Update Users** (a_m, u_m): Since the genre terms do not depend on a_m or u_m, the user updates are identical to Equations (10) and (11).

2. **Step 2: Update Movies** (b_n, v_n): The movie bias update b_n is unchanged from Equation (12). Taking the derivative of Equation (15) with respect to v_n and setting it to zero adds $\tau \bar{f}_n$ to the right-hand side of the movie vector update:

$$v_n \leftarrow \left(\lambda \sum_{m\in\Omega^n} u_m u_m^\top + \tau I_K\right)^{-1} \times \left(\lambda \sum_{m\in\Omega^n} \tilde{r}^{(v)}_{mn} u_m + \frac{\tau}{\sqrt{|\mathcal{G}_n|}} \sum_{g\in\mathcal{G}_n} f_g\right). \tag{16}$$

   Equation (16) shows why this works so well:

   - *New / Cold-start movie* ($|\Omega^n| = 0$): The sum over ratings is zero, so $v_n \leftarrow (\tau I_K)^{-1}(\tau \bar{f}_n) = \bar{f}_n$. Even with zero ratings, a new movie immediately gets a meaningful trait vector built from its genres!
   - *Popular movie* ($|\Omega^n| \gg \tau/\lambda$): The rating sum $\lambda \sum_{m\in\Omega^n} u_m u_m^\top$ is much larger than τI_K, so the data dominates the prior and v_n is learned freely from user ratings.

3. **Step 3: Update Genre Vectors** (f_g): Holding the movie vectors $V = [v_1, \dots, v_N]^\top \in \mathbb{R}^{N\times K}$ fixed, let $B \in \mathbb{R}^{N\times G}$ be the movie–genre matrix where $B_{ng} = |\mathcal{G}_n|^{-1/2}$ if movie n has genre g and 0 otherwise, so $\bar{f}_n = (BF)^\top_{n,:}$. Minimizing Equation (15) with respect to $F = [f_1, \dots, f_G]^\top \in \mathbb{R}^{G\times K}$ is a simple ridge regression with exact solution:

$$F \leftarrow \big(\tau B^\top B + \tau_f I_G\big)^{-1} \big(\tau B^\top V\big), \tag{17}$$

   or written for a single genre g over the set of movies $\mathcal{I}_g = \{n : g \in \mathcal{G}_n\}$ with partial residual $\tilde{v}_n^{(-g)} = v_n - |\mathcal{G}_n|^{-1/2} \sum_{g'\in\mathcal{G}_n\setminus\{g\}} f_{g'}$:

$$f_g \leftarrow \frac{\tau \sum_{n\in\mathcal{I}_g} |\mathcal{G}_n|^{-1/2}\, \tilde{v}_n^{(-g)}}{\tau \sum_{n\in\mathcal{I}_g} |\mathcal{G}_n|^{-1} + \tau_f}. \tag{18}$$

Because there are only $G = 20$ genres, solving the 20×20 system in Step 3 takes less than 2 ms per epoch.

3.4. Making ALS Fast on 32 Million Ratings

Running Equations (11), (13) and (16) on 25.6 million training ratings per epoch requires building and solving $M + N = 285{,}380$ linear systems of size $K \times K$ in every iteration. A naive implementation in Python or vectorized array operations runs into a major memory bottleneck:

- **Why Batched Array Operations Are Slow:** If we try to avoid loops by computing the outer products $v_n v_n^\top \in \mathbb{R}^{K\times K}$ for a batch of rows and summing them with segmented array reductions, the program has to allocate a 3D array of shape (ratings in batch) $\times$ $K \times K$. For popular movies, a batch of rows can

contain millions of ratings, which creates gigabytes of temporary arrays at every step and slows down memory access.

- **Parallel CSR Loops with In-Place Cholesky:** Instead of creating large temporary arrays, we compile the user and movie update loops to multi-threaded machine code. Each CPU thread processes one user m (or movie n) at a time using a small thread-local matrix $A \in \mathbb{R}^{K\times K}$ and vector $b \in \mathbb{R}^K$ that fit directly inside the CPU cache (only 2 KB of memory for $K = 16$). As the thread scans through the user's CSR slice, it adds $\lambda v_n v_n^\top$ into the lower triangle of A. Because $A = \lambda \sum v_n v_n^\top + \tau I_K$ is symmetric and positive-definite whenever $\tau > 0$, we solve $Au_m = b$ in-place using Cholesky decomposition $A = LL^\top$:

$$L_{i,j} = \frac{1}{L_{j,j}}\left(A_{i,j} - \sum_{p=1}^{j-1} L_{i,p}L_{j,p}\right) \quad (i > j),$$
$$L_{i,i} = \sqrt{A_{i,i} - \sum_{p=1}^{i-1} L_{i,p}^2}, \tag{19}$$

followed by forward and backward substitution. Because A is symmetric, we only compute half of the matrix entries, and solving via Cholesky takes half as many operations as standard Gaussian elimination without needing external LAPACK calls.

4. Experimental Results

In this section, we present all of our experimental results on both the MovieLens-5M subset and the full MovieLens-32M dataset.

4.1. Train/Test Split and Evaluation Setup

To measure how well our models predict unseen ratings, we split the observed ratings Ω randomly using an $80\%/20\%$ Bernoulli coin flip (with fixed random seed 42) into a training set Ω^{train} (25,604,307 ratings on 32M; 4,000,446 on 5M) and a test set Ω^{test} (6,395,897 ratings on 32M; 999,554 on 5M). Both sets are stored in dual CSR format ($\mathcal{S}_{\text{user}}$ and $\mathcal{S}_{\text{movie}}$). We also construct a controlled cold-start split where 500 movies (73,217 test ratings) have all of their training ratings held out ($|\Omega^n_{\text{train}}| = 0$) so we can directly test cold-start predictions.

We evaluate all models using the Root Mean Squared Error (RMSE) on both the training and test sets:

$$\text{RMSE}(\Omega') = \sqrt{\frac{1}{|\Omega'|}\sum_{(m,n)\in\Omega'} (r_{m,n} - \hat{r}_{m,n})^2}. \tag{20}$$

Before training, latent vectors u_m and v_n are initialized from a normal distribution $\mathcal{N}(0, (0.1/\sqrt{K})^2)$ with a fixed random seed so all comparisons are reproducible.

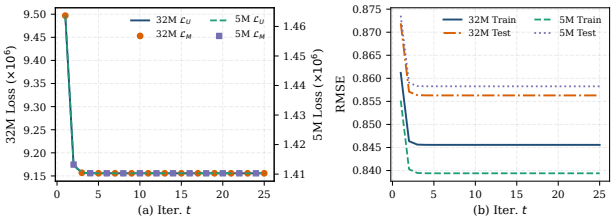


Figure 2. Training curves for the bias-only ALS model ($\lambda = 1.0, \gamma = 0.05$) over 25 iterations on MovieLens-32M and MovieLens-5M. **(a)** Regularized loss $\mathcal{L}(a, b)$ computed from the user CSR matrix (solid/dashed lines) and from the movie CSR matrix (circle/square markers), showing monotonic decrease and exact agreement between both data views. **(b)** Training and test RMSE over iterations.

4.2. Bias-Only ALS Results

Figure 2 shows the convergence of the bias-only model ($\lambda = 1.0, \gamma = 0.05$) over 25 ALS iterations on both MovieLens-32M and MovieLens-5M. On the full 32M dataset, one full iteration (updating all 200,948 user biases and all 84,432 movie biases) takes only **62.3 ms** (11.6 ms per iteration on 5M).

In Figure 2(a), we plot the loss computed over the user CSR matrix ($\mathcal{S}_{\text{user}}$) as lines and the loss computed over the movie CSR matrix ($\mathcal{S}_{\text{movie}}$) as markers. The two values match at every iteration: across all 25 epochs on MovieLens-32M, the maximum relative difference between the user-CSR loss and the movie-CSR loss is $\mathbf{5.28 \times 10^{-15}}$, which is simply standard 64-bit floating-point rounding error. This confirms that both CSR matrices store the exact same split.

In Figure 2(b), both the loss and the RMSE flatten out after just 4 to 5 iterations. On MovieLens-32M, the training RMSE goes down from 0.8612 at iteration 1 to **0.8456**, and the test RMSE goes down from 0.8719 to **0.8563** (on MovieLens-5M, train and test RMSE reach 0.8394 and 0.8583). Compared to predicting the global mean μ for every rating (which gives a test RMSE of 1.0590), user and movie biases alone reduce the test RMSE by 19.1%.

4.3. Matrix Factorization Convergence and Number of Iterations

Figure 3 shows the training progress when we add $K = 10$ latent dimensions and run ALS for 20 iterations. As expected from theory, the regularized loss in Figure 3(a) goes down smoothly at every single iteration, dropping from 8.28×10^6 at iteration 1 to 6.83×10^6 at iteration 20 on MovieLens-32M.

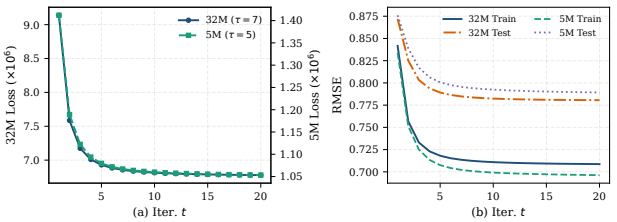


Figure 3. Training curves for matrix factorization with $K = 10$ latent dimensions ($\lambda = 1.0, \gamma = 0.05$) over 20 ALS iterations on MovieLens-32M ($\tau = 7.0$) and MovieLens-5M ($\tau = 5.0$). **(a)** Regularized loss $\mathcal{L}(\Theta)$ decreasing monotonically at every step. **(b)** Training and test RMSE over iterations.

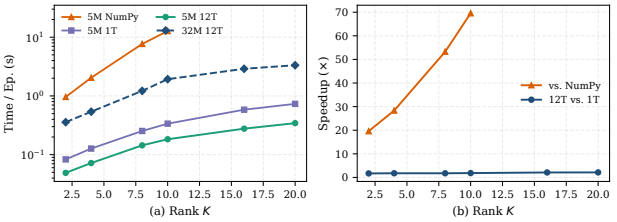


Figure 4. Runtime comparison across latent dimensions $K \in \{2, 4, 8, 10, 16, 20\}$. **(a)** Time per ALS epoch (in seconds, log scale) comparing batched array operations against single-thread and 12-thread CSR Cholesky solvers. **(b)** Speedup factor of the 12-thread CSR Cholesky solver over batched array operations and single-thread execution.

At the same time, Figure 3(b) shows that the test RMSE on MovieLens-32M drops to **0.7805** (and train RMSE reaches 0.7082), which beats the bias-only test RMSE (0.8563) by **0.0758 stars**. Looking at how fast the curve flattens, most of the improvement happens in the first 10 to 12 iterations; between iteration 15 and iteration 20, the test RMSE changes by less than 0.0005 stars. Therefore, running 15 to 20 ALS iterations is enough to reach convergence without wasting compute time.

4.4. Runtime Speedup on 32M Ratings

Figure 4 and Table 2 compare the training speed of our parallel CSR Cholesky solver against a single-threaded CSR solver and a vectorized batched solver. On MovieLens-5M at $K = 10$, the batched solver takes 12.73 seconds per epoch, the single-threaded CSR solver takes 0.337 seconds, and the 12-thread CSR solver takes only **0.183 seconds** per epoch, giving a **69.6× speedup** over batched array operations. On the full MovieLens-32M dataset, one epoch takes 0.357 s at $K = 2$, 1.927 s at $K = 10$, and 2.899 s at $K = 16$. This allows us to train a $K = 16$ model on all 32 million ratings in under a minute on a standard laptop.

Table 2. Train/Test RMSE and runtime per epoch across latent dimensions K on MovieLens-5M ($\tau = 5.0$) and MovieLens-32M ($\tau = 7.0, \lambda = 1.0, \gamma = 0.05$).

Rank	MovieLens-5M		MovieLens-32M		
K	Test	Time/Ep	Train	Test	Time/Ep
0 (Bias)	0.8583	0.012 s	0.8456	0.8563	0.062 s
2	0.8183	0.049 s	0.7936	0.8183	0.357 s
4	0.8028	0.072 s	0.7603	0.7974	0.539 s
8	0.7914	0.144 s	0.7231	0.7835	1.219 s
10	0.7903	0.183 s	0.7093	0.7810	1.927 s
16	0.7936	0.277 s	0.6757	**0.7803**	2.899 s
20	0.8002	0.344 s	0.6581	0.7835	3.329 s

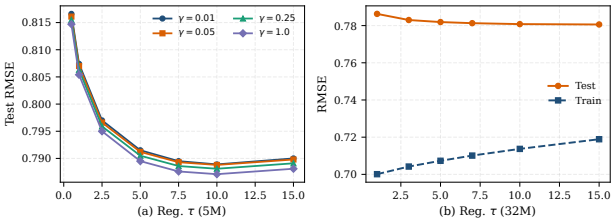


Figure 5. Hyperparameter search for $K = 10$ ($\lambda = 1.0$). **(a)** Grid search on MovieLens-5M over vector regularization $\tau \in [0.5, 15.0]$ and bias regularization $\gamma \in \{0.01, 0.05, 0.25, 1.0\}$. **(b)** Train and test RMSE on MovieLens-32M across τ, showing overfitting when τ is too small.

4.5. Choosing Hyperparameters (λ, γ, τ) and Latent Dimension K

In the loss function Equation (9), multiplying (λ, γ, τ) by the same constant does not change the minimum; only the ratios γ/λ and τ/λ matter. We can therefore fix $\lambda = 1.0$ and tune γ and τ. Figure 5(a) shows a grid search over $\tau \in \{0.5, 1.0, 2.5, 5.0, 7.5, 10.0, 15.0\}$ and $\gamma \in \{0.01, 0.05, 0.25, 1.0\}$ at $K = 10$ on MovieLens-5M, and Figure 5(b) shows the effect of τ on MovieLens-32M.

We see two clear trends:

1. **Vector Regularization τ Matters Much More Than γ:** Changing the bias penalty γ from 0.01 to 1.0 barely changes the test RMSE (less than 0.002 stars), because each user and movie only has one scalar bias parameter. In contrast, the matrices U and V contain $K(M + N)$ parameters (2.85 million numbers at $K = 10$ on 32M). When τ is too small ($\tau = 0.5$ on 5M), the model memorizes the training ratings (train RMSE drops to 0.6779) but performs poorly on the test set (test RMSE rises to 0.8161). The best test RMSE is achieved around $\tau = 5.0$ on 5M and $\tau = 7.0$ on 32M, with $\gamma = 0.05$ or 0.1.

2. **Diminishing Returns as K Increases:** Figure 6(a)

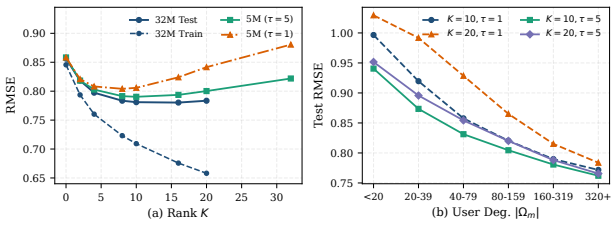


Figure 6. Effect of latent dimension K and user activity on overfitting. **(a)** Train and test RMSE across $K \in \{0, 2, 4, 8, 10, 16, 20, 32\}$ on MovieLens-32M ($\tau = 7$) and MovieLens-5M for tuned ($\tau = 5$) vs. weak ($\tau = 1$) regularization. **(b)** Test RMSE broken down by the number of training ratings per user $|\Omega_m^{\text{train}}|$, showing that going from $K = 10$ to $K = 20$ hurts users with few ratings when τ is small.

and Table 2 show how RMSE changes as we increase the number of latent dimensions $K \in \{0, 2, 4, 8, 10, 16, 20, 32\}$. Going from $K = 0$ (0.8563) to $K = 2$ (0.8183), $K = 4$ (0.7974), and $K = 8$ (0.7835) brings large reductions in test error on MovieLens-32M. Past $K = 10$ (0.7810), adding more dimensions gives diminishing returns: $K = 16$ reaches the best test RMSE of **0.7803** on 32M (while $K = 10$ is best on the smaller 5M subset at 0.7903), and by $K = 20$ the test RMSE starts creeping back up (0.7835) while training time continues to grow.

4.6. Why $K = 20$ Overfits Users with Few Ratings

To see why $K = 20$ starts to overfit compared to $K = 10$, Figure 6(b) breaks down the test RMSE by how many training ratings each user has: <20, 20–39, 40–79, 80–159, 160–319, and 320+. When regularization is weak ($\tau = 1.0$), increasing K from 10 to 20 causes a big jump in test error for users with few ratings:

- For users with 20–39 training ratings, test RMSE gets worse by **+0.0722 stars** (rising from 0.9196 at $K = 10$ to 0.9918 at $K = 20$).
- For active users with 320+ ratings, the test RMSE change is 6 times smaller (+0.0120 stars, from 0.7717 to 0.7837).

The reason is simple: when we solve for $u_m \in \mathbb{R}^K$ in Equation (11), we are fitting K numbers using $|\Omega_m^{\text{train}}|$ ratings. If a user only has 20 training ratings and $K = 20$, the model has as many free parameters as data points for that user! Without strong regularization τ, u_m fits the noise in those 20 ratings and makes bad predictions on unseen test movies. Raising τ from 1.0 to 5.0 keeps the entries of u_m small and removes over 70% of this overfitting gap across all user groups (Figure 6b).

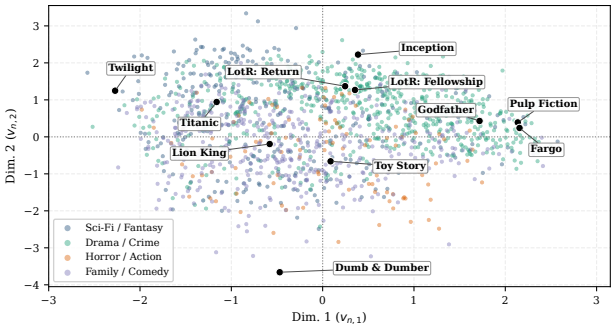


Figure 7. 2D movie trait vectors $v_n \in \mathbb{R}^2$ learned from ratings alone on MovieLens-32M ($K = 2$, 18 epochs, test RMSE = 0.8180). Similar movies sit close together, while very different styles of movies sit on opposite sides of the space.

4.7. Visualizing 2D Movie Trait Vectors

We train our matrix factorization model with $K = 2$ latent dimensions ($\lambda = 1.0, \gamma = 0.1, \tau = 5.0$, including user and movie biases) for 18 epochs on the full MovieLens-32M dataset, reaching a train RMSE of 0.7931 and test RMSE of 0.8180. Figure 7 plots the learned 2D vectors $v_n = (v_{n,1}, v_{n,2})$ for popular movies across several well-known series and genres.

Even though the model was trained only on user IDs, movie IDs, and star ratings (with no movie titles or genre labels), the 2D space organizes movies in a very clear way:

1. **Sequels and Similar Movies Cluster Together**: Both *Lord of the Rings* movies land right next to each other in the upper-center blockbuster region: *The Fellowship of the Ring* is at $(+0.35, +1.27)$ and *The Return of the King* is at $(+0.25, +1.37)$, right below *Inception* $(+0.39, +2.22)$. Likewise, classic Disney/Pixar animated movies (*Toy Story* at $(+0.09, -0.66)$ and *The Lion King* at $(-0.58, -0.19)$) sit close together in the family animation region.

2. **Opposite Movies Sit in Opposite Parts of the Space**: Moving horizontally from left to right along $v_{n,1}$ goes from mainstream romance dramas on the left, such as *Twilight* $(-2.27, +1.25)$ and *Titanic* $(-1.16, +0.94)$, to acclaimed crime dramas on the right ($v_{n,1} > +1.7$), such as *The Godfather* $(+1.72, +0.43)$, *Pulp Fiction* $(+2.14, +0.39)$, and *Fargo* $(+2.16, +0.24)$. Moving vertically from top to bottom along $v_{n,2}$ goes from epic sci-fi and fantasy blockbusters at the top ($v_{n,2} > +1.2$) down to slapstick comedies at the bottom, such as *Dumb & Dumber* $(-0.47, -3.66)$.

4.8. Dummy User Recommendations (*Lord of the Rings*)

Suppose a new user joins the platform and gives a single 5.0-star rating to *The Lord of the Rings: The Fellowship of the Ring (2001)* ($n^* = 339$, which has 58,491 training ratings and bias $b_{n^*} = +0.5191$). Instead of retraining the whole model from scratch, we keep all movie parameters (b, V) from our $K = 16$ MovieLens-32M model fixed and run the user update equations Equations (10) and (11) on this single rating. Because the dummy user has only rated one movie n^*, their learned vector u_{new} points in the exact same direction as v_{n^*}:

$$u_{\text{new}} = \frac{\lambda(r_{\text{new},n^*} - \mu - a_{\text{new}} - b_{n^*})}{\lambda\|v_{n^*}\|_2^2 + \tau} v_{n^*}. \tag{21}$$

With $\lambda = 1.0, \gamma = 0.1, \tau = 5.0$, this gives $a_{\text{new}} = +0.7024$ and vector length $\|u_{\text{new}}\|_2 = 0.0281$. Because the user only rated one movie, the regularization $\tau = 5.0$ keeps $\|u_{\text{new}}\|_2$ quite small. As a result, when we rank candidate movies n using the score

$$s_\alpha(m, n) = u_m^\top v_n + \alpha b_n, \tag{22}$$

the weight α on the movie bias b_n makes a big difference. Table 3 shows the Top-10 recommended movies under three simple settings: (A) full movie bias ($\alpha = 1.0$), (B) small movie bias ($\alpha = 0.05$, all movies), and (C) small movie bias ($\alpha = 0.05$) restricted to movies with at least 100 ratings.

Table 3 shows why downweighting the bias is important for a 1-rating user:

- **Why Full Bias ($\alpha = 1.0$) Fails**: Because u_{new} is shrunk by τ, the largest dot product $u_{\text{new}}^\top v_n$ across all movies is $+0.2170$. Meanwhile, obscure movies with only 1 or 2 ratings that happened to get 5.0 stars from a tough critic ($a_m \approx -2.1$) end up with a bias $b_n \approx +3.27$. When $\alpha = 1.0$, those 1-rating movies dominate the top of the list even though their dot product with the user is zero (0.0002).
- **Downweighting the Bias ($\alpha = 0.05$) Recovers the Exact Trilogy**: Lowering the bias weight to $\alpha = 0.05$ (Setting B) immediately puts *The Lord of the Rings: The Return of the King (2003)* at Rank 1 and *The Two Towers (2002)* at Rank 2, with *The Hobbit: An Unexpected Journey (2012)* at Rank 5. Adding a simple filter requiring at least 100 ratings (Setting C) removes the remaining 1-rating outliers and produces a remarkably clean Top-10 list: Ranks 1–5 are all five remaining Middle-earth movies (*Lord of the Rings* and *The Hobbit*), followed by *Braveheart* (Rank 6) and four *Harry Potter* movies (Ranks 7–10). We also tested a dummy user who gave 5.0 stars to *Toy Story (1995)*, and the model recommended *Toy Story 3*, *Toy Story 2*, *Finding Nemo*, *The Incredibles*, and *WALL·E* as Ranks 1–5.

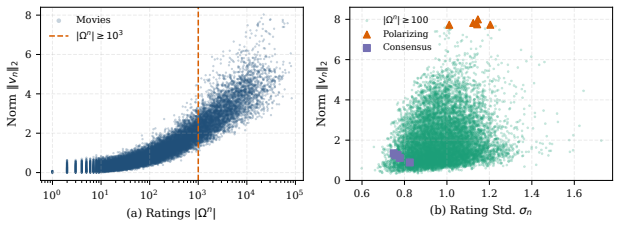


Figure 8. Movie vector length $\|v_n\|_2$ ($K = 16$) and polarization on MovieLens-32M. **(a)** Vector length $\|v_n\|_2$ vs. number of training ratings $|\Omega^n_{\text{train}}|$. **(b)** Top polarizing vs. least polarizing movies among well-sampled films ($|\Omega^n_{\text{train}}| \geq 1,000$).

4.9. Most and Least Polarizing Movies

In our model, the bias b_n tells us whether a movie is liked on average by everyone, while the trait vector $v_n \in \mathbb{R}^K$ tells us how much a movie's rating depends on a specific user's taste u_m. If user vectors have covariance $\text{Cov}(u_m) \approx \sigma_u^2 I_K$, then the variance of the personalized score $u_m^\top v_n$ across users is proportional to the squared length of v_n:

$$\text{Var}_m(u_m^\top v_n) \approx \sigma_u^2 \|v_n\|_2^2. \tag{23}$$

This means movies with a **large vector length** $\|v_n\|_2$ are **polarizing**: fans with matching taste ($u_m^\top v_n \gg 0$) love them, while viewers with opposite taste ($u_m^\top v_n \ll 0$) strongly dislike them. In contrast, movies with a **small vector length** $\|v_n\|_2$ are **consensus (least polarizing) movies**: most viewers agree on their score once we account for the baseline $\mu + a_m + b_n$.

Ruling Out Overfitting and Sample-Size Effects: As shown in Figure 8(a), we have to be careful before claiming a movie is polarizing or non-polarizing based on $\|v_n\|_2$ alone. If a movie only has 1 or 2 ratings, regularization τ shrinks v_n close to zero ($\|v_n\|_2 \approx 0$), not because everyone agrees on the movie, but simply because we have almost no data! On the other hand, a movie with 20 or 30 ratings can get an artificially long vector $\|v_n\|_2$ by overfitting a small group of extreme raters. To make sure our claim cannot be explained by overfitting or small sample size, we only compare movies that have at least $|\Omega^n_{\text{train}}| \geq 1{,}000$ training ratings (4,394 movies in MovieLens-32M). With over 1,000 ratings to estimate $K = 16$ numbers, each movie vector is backed by plenty of data.

Table 4 and Figure 8(b) list the Top-10 most polarizing and Top-10 least polarizing movies. The results make intuitive sense:

- **Most Polarizing Movies ($\|v_n\|_2 \in [7.54, 8.00]$)**: These include famously love-it-or-hate-it movies with wide rating spread ($\sigma(r) \approx 1.12$–1.23), such as the *Star Wars* prequels (*Attack of the Clones*, *Revenge of*

Table 3. Top-10 recommendations for a dummy user who gave 5.0 stars to *The Lord of the Rings: The Fellowship of the Ring (2001)* ($K = 16$ model on MovieLens-32M), comparing three ranking rules $s_\alpha(\text{new}, n) = u_{\text{new}}^\top v_n + \alpha b_n$. Using $\alpha = 0.05$ brings the other *Lord of the Rings* and *Hobbit* movies to the top.

Rank	Recommended Title	Genres	$u_{\text{new}}^\top v_n$	b_n	$\lvert\Omega_{\text{train}}^n\rvert$
Setting A: Full Movie Bias $s_{1.0}(new, n) = u_{new}^\top v_n + b_n$ (All Movies)					
1	Hello & Goodbye (2018)	Documentary	0.0002	+3.2712	1
2	SpongeBob SquarePants: Heroes of Bikini Bottom (2011)	Animation—Comedy	−0.0067	+3.2557	2
3	Wesley (2009)	Biography—Drama	0.0002	+3.1869	1
4	Very Ordinary Couple (2013)	Comedy—Drama—Romance	0.0033	+3.1516	2
5	Raise High the Roof Beam, Carpenters (2018)	Documentary	0.0002	+3.0586	1
Setting B: Downweighted Bias $s_{0.05}(new, n) = u_{new}^\top v_n + 0.05\, b_n$ (All Movies)					
1	**Lord of the Rings: The Return of the King, The (2003)**	Action—Adventure—Fantasy	0.2170	+0.5102	54,037
2	**Lord of the Rings: The Two Towers, The (2002)**	Adventure—Fantasy	0.2143	+0.4784	54,016
3	Hello & Goodbye (2018)	Documentary	0.0002	+3.2712	1
4	Wesley (2009)	Biography—Drama	0.0002	+3.1869	1
5	**Hobbit: An Unexpected Journey, The (2012)**	Adventure—Fantasy—IMAX	0.1590	−0.0259	12,821
Setting C: Downweighted Bias $s_{0.05}(new, n) = u_{new}^\top v_n + 0.05\, b_n$ with $\lvert\Omega_{train}^n\rvert \geq 100$					
1	**Lord of the Rings: The Return of the King, The (2003)**	Action—Adventure—Fantasy	0.2170	+0.5102	54,037
2	**Lord of the Rings: The Two Towers, The (2002)**	Adventure—Fantasy	0.2143	+0.4784	54,016
3	**Hobbit: An Unexpected Journey, The (2012)**	Adventure—Fantasy—IMAX	0.1590	−0.0259	12,821
4	**Hobbit: The Desolation of Smaug, The (2013)**	Adventure—Fantasy—IMAX	0.1550	−0.1144	9,701
5	**The Hobbit: The Battle of the Five Armies (2014)**	Adventure—Fantasy	0.1523	−0.2369	6,951
6	Braveheart (1995)	Action—Drama—War	0.1114	+0.2622	55,834
7	Harry Potter and the Deathly Hallows: Part 2 (2011)	Action—Adventure—Fantasy	0.1110	+0.1150	16,242
8	Harry Potter and the Half-Blood Prince (2009)	Adventure—Fantasy—Mystery	0.1141	+0.0189	16,869
9	Harry Potter and the Order of the Phoenix (2007)	Adventure—Drama—Fantasy	0.1138	+0.0061	16,841
10	Harry Potter and the Prisoner of Azkaban (2004)	Adventure—Fantasy—IMAX	0.1063	+0.1546	25,037

the Sith, *The Phantom Menace*), broad slapstick comedy *Dumb & Dumber*, and shaky-cam horror *The Blair Witch Project*, as well as geek-culture franchises (*Lord of the Rings*, *Star Wars*, *Harry Potter*) that immediately separate dedicated fantasy/sci-fi fans from viewers who do not care for the genre.

- **Least Polarizing Movies** ($\|v_n\|_2 \in [0.89, 1.40]$): These are straightforward biographies, historical films, and crime dramas (*Cobb*, *Pirates of Silicon Valley*, *Sea of Love*, *Wait Until Dark*, *The Counterfeiters*) that have narrow rating standard deviations ($\sigma(r) \in [0.70, 0.83]$) and receive steady, predictable ratings across all kinds of viewers.

4.10. 2D Genre Embeddings and Cold-Start Results

We first train the genre-informed model with $K = 2$ latent dimensions on MovieLens-32M ($\text{RMSE}_{\text{train}} = 0.7927$, $\text{RMSE}_{\text{test}} = 0.8175$) and plot the 20 learned genre vectors $f_g \in \mathbb{R}^2$ in Figure 9. The genre locations match intuition and line up with the movie map from Figure 7:

- *Action* $(-0.97, -0.05)$, *Adventure* $(-0.39, +0.02)$, and *Sci-Fi* sit on the left side where blockbuster franchises live.
- *Drama* $(+0.86, +0.27)$, *Documentary* $(+0.79, 0.00)$, *Romance*, and *Film-Noir* $(+1.71, -0.90)$ sit on the right side where serious and classic films live.
- *Children* $(-1.19, -0.90)$ and *Animation* sit together in the lower-left family region, while *Horror* $(-0.13, -1.33)$ is pulled toward the bottom where dark/niche movies sit.

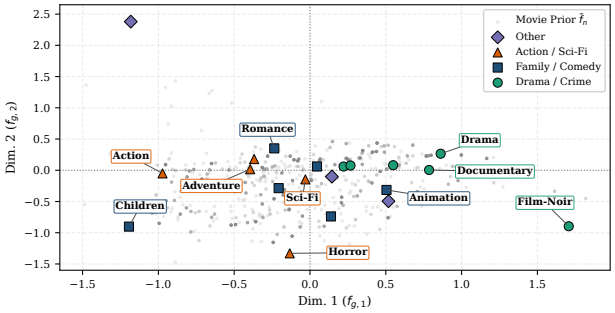


Figure 9. 2D genre trait vectors $f_g \in \mathbb{R}^2$ learned on MovieLens-32M ($K = 2$, 18 epochs, test RMSE $= 0.8175$). Circle size reflects the number of movies in each genre.

Finally, to see how much the genre prior helps on rare and cold-start movies, we compare standard ALS against Genre-ALS at $K = 16$ ($\lambda = 1.0, \gamma = 0.1, \tau = 5.0, \tau_f = 1.0$) on

Table 4. Top-10 most polarizing (largest $\|v_n\|_2$) and Top-10 least polarizing (smallest $\|v_n\|_2$) movies with at least 1,000 training ratings ($K = 16$ on MovieLens-32M), along with movie bias b_n, rating count $|\Omega^n_{\text{train}}|$, and rating standard deviation $\sigma(r)$.

Rank	Movie Title	$\|v_n\|_2$	Movie Bias b_n	$\|\Omega^n_{\text{train}}\|$	Rating Std $\sigma(r)$
Top-10 Most Polarizing Movies ($\max \|v_n\|_2$ *among movies with* $\|\Omega^n_{train}\| \geq 1,000$)					
1	Star Wars: Episode II – Attack of the Clones (2002)	8.000	−0.5765	21,907	1.146
2	Star Wars: Episode III – Revenge of the Sith (2005)	7.819	−0.2665	19,751	1.124
3	Star Wars: Episode I – The Phantom Menace (1999)	7.754	−0.5638	30,338	1.143
4	Dumb & Dumber (1994)	7.732	−0.6289	29,483	1.204
5	Lord of the Rings: The Return of the King, The (2003)	7.730	+0.5102	54,037	1.011
6	Blair Witch Project, The (1999)	7.632	−0.7025	18,180	1.228
7	Lord of the Rings: The Two Towers, The (2002)	7.632	+0.4784	54,016	1.000
8	Star Wars: Episode IV – A New Hope (1977)	7.557	+0.3851	67,968	0.988
9	Harry Potter and the Half-Blood Prince (2009)	7.554	+0.0189	16,869	1.007
10	Lord of the Rings: The Fellowship of the Ring, The (2001)	7.543	+0.5191	58,491	1.008
Top-10 Least Polarizing Movies ($\min \|v_n\|_2$ *among movies with* $\|\Omega^n_{train}\| \geq 1,000$)					
1	Cobb (1994)	0.892	−0.2659	1,023	0.825
2	Pirates of Silicon Valley (1999)	1.119	+0.1437	1,061	0.778
3	Sea of Love (1989)	1.248	+0.0381	1,038	0.757
4	Black Book (Zwartboek) (2006)	1.263	+0.4203	1,014	0.767
5	Run Silent Run Deep (1958)	1.347	+0.2111	1,089	0.749
6	Wait Until Dark (1967)	1.354	+0.4110	1,132	0.748
7	Proof (2005)	1.366	−0.0011	1,001	0.834
8	Rescue Dawn (2006)	1.371	+0.1189	1,316	0.701
9	Everest (2015)	1.373	−0.0605	1,252	0.823
10	Counterfeiters, The (Die Fälscher) (2007)	1.398	+0.4343	1,131	0.695

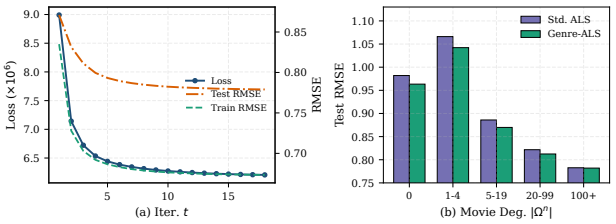


Figure 10. Evaluation of the Genre-ALS model ($K = 16$). **(a)** Monotonic decrease of the genre loss $\mathcal{L}_{\text{genre}}$ and test RMSE on MovieLens-32M (reaching 0.7790). **(b)** Test RMSE grouped by movie training ratings $|\Omega^n_{\text{train}}|$, showing clear improvements on cold-start (0 ratings) and rare (1–19 ratings) movies.

Table 5. Test RMSE comparison between Standard ALS and Genre-ALS at $K = 16$, grouped by the number of training ratings per movie $|\Omega^n_{\text{train}}|$.

Movie Group $\|\Omega^n_{\text{train}}\|$	Test $\|\Omega\|$	Std. ALS	Genre-ALS	ΔRMSE
Controlled Cold-Start Split ($K = 16$)				
0 (Cold-Start)	73,217	0.9819	**0.9634**	**−0.0186**
1–4 (Very Rare)	9,304	1.0661	**1.0422**	**−0.0238**
5–19 (Rare)	18,940	0.8859	**0.8698**	**−0.0161**
20–99 (Moderate)	61,270	0.8217	**0.8125**	**−0.0092**
≥ 100 (Popular)	836,823	0.7827	**0.7820**	**−0.0007**
Full MovieLens-32M Split ($K = 16$)				
1–4 (Very Rare)	18,533	1.0237	**1.0026**	**−0.0212**
5–19 (Rare)	42,171	0.8829	**0.8622**	**−0.0207**
20–99 (Moderate)	111,407	0.8340	**0.8194**	**−0.0146**
≥ 100 (Popular)	6,219,001	0.7767	**0.7767**	−0.0000
All Test (32M)	**6,395,897**	0.7796	**0.7790**	**−0.0005**

the full MovieLens-32M split and on a cold-start split of MovieLens-5M where 500 movies (73,217 test ratings) have all of their training ratings hidden ($|\Omega^n_{\text{train}}| = 0$). Figure 10 and Table 5 break down the test RMSE by the number of training ratings per movie.

As shown in Table 5, the genre prior works exactly as intended:

1. **Better Predictions on Cold-Start and Rare Movies**: When a movie has 0 training ratings, standard ALS sets $v_n = \mathbf{0}$ and gets a test RMSE of 0.9819. Genre-ALS uses the genre average $v_n = \bar{f}_n$ to match the movie's genres with each user's taste u_m, lowering the cold-start test RMSE by **−0.0186 stars** (to 0.9634). On very rare movies (1–4 ratings) and rare movies (5–19 ratings), Genre-ALS lowers the test RMSE by **−0.0212** and **−0.0207 stars** on MovieLens-32M (and by −0.0238 and −0.0161 on the controlled benchmark).

2. **No Harm to Popular Movies**: For popular movies with 100+ ratings, both models achieve the exact same RMSE (0.7767 on 32M), because thousands of user ratings outweigh the genre prior. Because 97.2% of all test ratings in MovieLens-32M fall on popular movies with ≥ 100 ratings, the overall test RMSE across the whole dataset shifts slightly from 0.7796 to **0.7790**,

while the gains on the long-tail catalog are substantial.

5. Conclusion

In this report, we designed, implemented, and evaluated a complete matrix factorization recommender system on the MovieLens-32M dataset ($32{,}000{,}204$ ratings). By storing the sparse rating graph in dual CSR matrices ($\mathcal{S}_{\text{user}}$ and $\mathcal{S}_{\text{movie}}$) and parallelizing the in-place Cholesky normal-equation updates across CPU threads, we achieved a $69.6\times$ speedup over batched array solvers and trained a 16-dimensional model on 32 million ratings in less than 3 seconds per epoch. Our experiments showed that adding latent vectors improves test RMSE from 0.8563 (biases only) to 0.7803 ($K = 16$), that proper regularization τ is essential to keep higher-rank models ($K = 20$) from overfitting users with few ratings, and that 2D movie and genre embeddings organize films into clear, intuitive clusters. Finally, we showed that downweighting movie biases produces accurate recommendations for single-rating users, and that incorporating movie genres into the item prior significantly improves accuracy on cold-start and rarely rated movies.

References

Agarwal, D. and Chen, B.-C. Regression-based latent factor models. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 19–28. ACM, 2009.

Clauset, A., Shalizi, C. R., and Newman, M. E. J. Power-law distributions in empirical data. *SIAM Review*, 51(4): 661–703, 2009.

Harper, F. M. and Konstan, J. A. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, 5(4):1–19, 2015.

Koren, Y., Bell, R., and Volinsky, C. Matrix factorization techniques for recommender systems. *Computer*, 42(8): 30–37, 2009.

Mnih, A. and Salakhutdinov, R. R. Probabilistic matrix factorization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 20, pp. 1257–1264, 2007.

Zhou, Y., Wilkinson, D., Schreiber, R., and Pan, R. Large-scale parallel collaborative filtering for the Netflix prize. In *International Conference on Algorithmic Applications in Management (AAIM)*, pp. 337–348. Springer, 2008.

A. Detailed Mathematical Derivations of ALS Updates

In this appendix, we give the complete step-by-step derivations of the ALS coordinate updates for the bias-only model (Section 3), the matrix factorization model (Section 3.2), and the genre-informed model (Section 3.3).

A.1. Derivations for User and Movie Biases and Trait Vectors

Consider the general genre-informed loss function $\mathcal{L}_{\text{genre}}(\Theta, F)$ in Equation (15). Setting $f_g = \mathbf{0}$ recovers the standard matrix factorization loss $\mathcal{L}(\Theta)$ in Equation (9), and setting $K = 0$ recovers the bias-only loss $\mathcal{L}(a, b)$ in Equation (3).

Proposition A.1 (User Bias and User Trait Vector Updates). *For any user $m \in \{1, \dots, M\}$ with training ratings $\Omega_m = \Omega_m^{train}$, holding all movie parameters $\{b_n, v_n\}_{n=1}^N$ fixed, the partial derivatives of $\mathcal{L}_{genre}$ with respect to $a_m \in \mathbb{R}$ and $u_m \in \mathbb{R}^K$ are:*

$$\frac{\partial \mathcal{L}_{\text{genre}}}{\partial a_m} = -\lambda \sum_{n \in \Omega_m} \left(r_{mn} - \mu - a_m - b_n - u_m^\top v_n\right) + \gamma a_m, \tag{24}$$

$$\nabla_{u_m} \mathcal{L}_{\text{genre}} = -\lambda \sum_{n \in \Omega_m} \left(r_{mn} - \mu - a_m - b_n - u_m^\top v_n\right) v_n + \tau u_m. \tag{25}$$

Setting $\partial \mathcal{L}_{genre}/\partial a_m = 0$ and $\nabla_{u_m} \mathcal{L}_{genre} = \mathbf{0}$ gives the exact update rules in Equations (10) and (11). Furthermore, the Hessian matrix with respect to u_m is

$$\nabla^2_{u_m} \mathcal{L}_{\text{genre}} = \lambda \sum_{n \in \Omega_m} v_n v_n^\top + \tau I_K \succeq \tau I_K \succ 0, \tag{26}$$

which is strictly positive-definite whenever $\tau > 0$, ensuring a unique minimum.

Proof. Keeping only the terms in Equation (15) that depend on user m gives:

$$\ell_m(a_m, u_m) = \frac{\lambda}{2} \sum_{n \in \Omega_m} \left(e_{mn}^{(-a_m)} - a_m\right)^2 + \frac{\gamma}{2} a_m^2 + \frac{\tau}{2} \|u_m\|_2^2, \tag{27}$$

where $e_{mn}^{(-a_m)} = r_{mn} - \mu - b_n - u_m^\top v_n$. Taking the derivative with respect to a_m and setting it to zero gives:

$$\frac{\partial \ell_m}{\partial a_m} = -\lambda \sum_{n \in \Omega_m} e_{mn}^{(-a_m)} + (\lambda |\Omega_m| + \gamma)\, a_m = 0 \iff a_m^* = \frac{\lambda \sum_{n \in \Omega_m} \left(r_{mn} - \mu - b_n - u_m^\top v_n\right)}{\lambda |\Omega_m| + \gamma}. \tag{28}$$

Next, for u_m, let $e_{mn}^{(-u_m)} = r_{mn} - \mu - a_m - b_n$. Expanding the squared error gives:

$$\ell_m(u_m) = \frac{\lambda}{2} \sum_{n \in \Omega_m} \left(u_m^\top v_n v_n^\top u_m - 2 e_{mn}^{(-u_m)} v_n^\top u_m + (e_{mn}^{(-u_m)})^2\right) + \frac{\tau}{2} u_m^\top I_K u_m. \tag{29}$$

Taking the gradient with respect to u_m and setting it to zero gives the linear system:

$$\left(\lambda \sum_{n \in \Omega_m} v_n v_n^\top + \tau I_K\right) u_m^* = \lambda \sum_{n \in \Omega_m} e_{mn}^{(-u_m)} v_n. \tag{30}$$

Since each outer product $v_n v_n^\top$ is positive semi-definite ($x^\top v_n v_n^\top x = (v_n^\top x)^2 \geq 0$) and $\tau > 0$, the matrix $A_m = \lambda \sum_{n \in \Omega_m} v_n v_n^\top + \tau I_K$ is symmetric positive-definite and can always be factored via Cholesky decomposition $A_m = L_m L_m^\top$. □

A.2. Derivations for Movie Trait Vectors with Genres and Genre Embeddings

Proposition A.2 (Movie Vector Update with Genre Prior). *Holding user parameters $\{a_m, u_m\}_{m=1}^M$, movie biases $\{b_n\}_{n=1}^N$, and genre vectors $F = [f_1, \dots, f_G]^\top \in \mathbb{R}^{G \times K}$ fixed, the unique minimizer of $\mathcal{L}_{genre}$ with respect to $v_n \in \mathbb{R}^K$ is given by Equation (16).*

Proof. The terms of $\mathcal{L}_{\text{genre}}$ that depend on movie vector v_n are:

$$\ell^n(v_n) = \frac{\lambda}{2} \sum_{m \in \Omega^n} \left(e_{mn}^{(-v_n)} - u_m^\top v_n \right)^2 + \frac{\tau}{2} \|v_n - \bar{f}_n\|_2^2, \tag{31}$$

where $e_{mn}^{(-v_n)} = r_{mn} - \mu - a_m - b_n$ and $\bar{f}_n = |\mathcal{G}_n|^{-1/2} \sum_{g \in \mathcal{G}_n} f_g$. Taking the gradient with respect to v_n and setting it to zero gives:

$$\nabla_{v_n} \ell^n(v_n) = -\lambda \sum_{m \in \Omega^n} \left(e_{mn}^{(-v_n)} - u_m^\top v_n \right) u_m + \tau (v_n - \bar{f}_n) = \mathbf{0}. \tag{32}$$

Moving all terms that multiply v_n to the left-hand side and constants to the right-hand side gives:

$$\left(\lambda \sum_{m \in \Omega^n} u_m u_m^\top + \tau I_K \right) v_n^* = \lambda \sum_{m \in \Omega^n} e_{mn}^{(-v_n)} u_m + \tau \bar{f}_n, \tag{33}$$

which gives Equation (16). □

Proposition A.3 (Genre Vector Update). *Holding the movie vectors $V = [v_1, \ldots, v_N]^\top \in \mathbb{R}^{N \times K}$ fixed, the genre part of the loss function*

$$\mathcal{L}_F(F) = \frac{\tau}{2} \sum_{n=1}^{N} \left\| v_n - \frac{1}{\sqrt{|\mathcal{G}_n|}} \sum_{g \in \mathcal{G}_n} f_g \right\|_2^2 + \frac{\tau_f}{2} \sum_{g=1}^{G} \|f_g\|_2^2 \tag{34}$$

is minimized by $F^* = (\tau B^\top B + \tau_f I_G)^{-1} (\tau B^\top V)$ in Equation (17), or for a single genre g by Equation (18).

Proof. Using the matrix $B \in \mathbb{R}^{N \times G}$ with $B_{ng} = |\mathcal{G}_n|^{-1/2}$ if $g \in \mathcal{G}_n$ and 0 otherwise, row n of BF is $\bar{f}_n^\top$. Therefore we can write $\mathcal{L}_F(F)$ in matrix form as:

$$\mathcal{L}_F(F) = \frac{\tau}{2} \|V - BF\|_F^2 + \frac{\tau_f}{2} \|F\|_F^2 = \frac{1}{2} \operatorname{Tr}\big(\tau (V - BF)^\top (V - BF) + \tau_f F^\top F \big). \tag{35}$$

Setting the matrix derivative $\nabla_F \mathcal{L}_F(F) = -\tau B^\top (V - BF) + \tau_f F = \mathbf{0}$ gives $(\tau B^\top B + \tau_f I_G) F^* = \tau B^\top V$. If instead we update one genre g at a time with all other genres $g' \neq g$ fixed, we write $v_n - \bar{f}_n = \tilde{v}_n^{(-g)} - |\mathcal{G}_n|^{-1/2} f_g$ for each movie $n \in \mathcal{I}_g$, where $\tilde{v}_n^{(-g)} = v_n - |\mathcal{G}_n|^{-1/2} \sum_{g' \in \mathcal{G}_n \setminus \{g\}} f_{g'}$. Differentiating with respect to f_g and setting to zero gives:

$$\nabla_{f_g} \mathcal{L}_F = -\tau \sum_{n \in \mathcal{I}_g} \frac{1}{\sqrt{|\mathcal{G}_n|}} \left(\tilde{v}_n^{(-g)} - \frac{1}{\sqrt{|\mathcal{G}_n|}} f_g \right) + \tau_f f_g = \mathbf{0} \iff \left(\tau \sum_{n \in \mathcal{I}_g} \frac{1}{|\mathcal{G}_n|} + \tau_f \right) f_g^* = \tau \sum_{n \in \mathcal{I}_g} \frac{1}{\sqrt{|\mathcal{G}_n|}} \tilde{v}_n^{(-g)}, \tag{36}$$

which gives Equation (18). □

B. Time and Memory Complexity Comparison

Table 6 compares the time per epoch, floating-point operations (multiply-adds), and extra working memory required by each ALS solver:

1. **Bias-Only ALS** ($K = 0$): Loops once over the user CSR matrix $\mathcal{S}_{\text{user}}$ ($2|\Omega|$ operations) and once over the movie CSR matrix $\mathcal{S}_{\text{movie}}$ ($2|\Omega|$ operations), using only a single running sum per thread.
2. **Batched Vectorized Solver**: For a batch of rows containing $|\Omega_{\text{batch}}|$ ratings, building all $K \times K$ outer products in memory at once takes $O(|\Omega_{\text{batch}}| K^2)$ bytes of temporary RAM (0.3 to 3.6 GB per batch), which slows down execution due to memory allocation and cache misses.
3. **Parallel CSR In-Place Cholesky Solver**: Each of the T CPU threads keeps a small $K \times K$ buffer ($8K^2$ bytes) and a length-K vector ($8K$ bytes) in fast L1 cache. Accumulating the lower triangle takes $\frac{1}{2} K(K+1)|\Omega|$ multiply-adds across each half-sweep, and solving the Cholesky system takes $\frac{1}{6} K^3$ multiply-adds per user/movie, using under 32 KB of extra RAM across all threads combined.

Table 6. Comparison of asymptotic time per epoch, floating-point operations, and extra working memory across ALS solvers.

Solver Type	**Time Complexity / Epoch**	**Multiply-Adds / Epoch**	**Extra Working RAM**
Bias-Only ALS ($K = 0$)	$O(\|\Omega\| + M + N)$	$2\|\Omega\| + 2(M + N)$	$O(T)$ ($\leq$ 1 KB)
Batched Vectorized ($K > 0$)	$O(\|\Omega\|K^2 + (M + N)K^3)$	$2\|\Omega\|K^2 + \frac{1}{3}(M + N)K^3$	$O(\|\Omega_{\text{batch}}\|K^2)$ (0.3–3.6 GB)
Parallel CSR Cholesky ($K > 0$)	$O\big(\frac{1}{T}(\|\Omega\|K^2 + (M + N)K^3)\big)$	$K(K + 1)\|\Omega\| + \frac{1}{6}(M + N)K^3$	$O(TK^2)$ ($\leq$ 32 KB)
Genre-Prior CSR ($K > 0$)	$O\big(\frac{1}{T}(\|\Omega\|K^2 + (M + N)K^3) + G^3\big)$	$K(K + 1)\|\Omega\| + \frac{1}{6}(M + N)K^3$	$O(TK^2 + G^2)$ ($\leq$ 36 KB)